Allow Attributes on Template Explicit Instantiations

Document: P0537R0
Date: 2016-08-23
Project: ISO/IEC JTC1 SC22 WG21 Programming Language C++
Audience: Evolution Working Group
Author: Matthew Woehlke (mwoehlke.floss@gmail.com)

Abstract

This proposal recommends to remove the prohibition against attributes on a template explicit instantiation.

(Note: references made to the existing draft standard are made against N4606.)

Contents

Problem

When creating a C++ library, it is typically necessary to annotate in some manner those functions which should be made available to consumers of the library (as opposed to being internal to the library itself). On Windows, this takes the form of `__declspec(dllexport)`. On ELF platforms, when using hidden visibility, this may look like `__attribute__((visibility("default")))`. Since the proper decoration is dependent on multiple factors — target platform being the most obvious, but in the case of `__declspec`, the decoration must differ depending on whether the library is being built or consumed — most libraries will define a preprocessor "export decoration symbol" to simplify 'decorating' functions to be exported.

Combined with templates, where the definition of a template function may be internal, but certain instantiations need to be made available to users of the library, one can image code like so:

```
// A template function
template <typename T> T foo(T*)
{
  // ...
}

// An exported explicit instantiation
template FOO_EXPORT int foo<int>(int*);
```

Since attributes were introduced in C++11, there has been activity toward standardizing the mechanisms for export decoration. In particular, GCC and Clang support the use of `[[gnu::visibility("default")]]` as a replacement for `__attribute__((visibility("default")))`. This is an obvious improvement: it is shorter to write, and compilers that don't understand the C++11 attribute may ignore it, rather than raising a syntax error as would be the case if the old form were used with a compiler that does not support it.

Astute readers may have spotted the problem by now: *explicit instantiations forbid attributes* ([dcl.attr.grammar]¶5). This means that the above example cannot use a C++11 attribute on a conforming compiler; indeed, there is no way to write strictly conforming code that also specifies that the instantiation should be exported.

Proposal

Presently, C++ forbids attributes on template explicit instantiations. We suspect that this limitation was imposed in the belief

that there are no reasonable attributes that might be applied to such. However, as we have shown above, this is not the case.

We propose to remove this restriction. The standard already allows that an unreasonable attribute may be rejected, so this change by itself does not introduce the ability to do undesirable things. However, it would add freedom to compiler vendors, by allowing them to accept reasonable attributes (e.g. vendor-specific attributes such as export annotation) applied to explicit instantiations. In particular, this change addresses an issue which prevents authors from fully switching to C++11 attributes for export decoration.

We believe that the utility of this change is self evident, as exemplified above, and that it matches programmer expectations. We are also not aware of any serious reasons for the restriction to exist.

Proposed Wording

(Proposed changes are specified relative to N4606.)

In [dcl.attr.grammar]¶5, make the following change:

> Each *attribute-specifier-seq* is said to *appertain* to some entity or statement, identified by the syntactic context where it appears (Clause 6, Clause 7, Clause 8). If an *attribute-specifier-seq* that appertains to some entity or statement contains an *attribute* that is not allowed to apply to that entity or statement, the program is ill-formed. If an *attribute-specifier-seq* appertains to a friend declaration (11.3), that declaration shall be a definition. ~~No *attribute-specifier-seq* shall appertain to an explicit instantiation (14.7.2).~~

Implementation

At least GCC 4.8 and 6.1 (and presumably all intervening versions) do not implement this restriction and allow attributes — at least the `gnu::visibility` attribute — to be applied to explicit instantiations.

Acknowledgments

We wish to thank Richard Smith for pointing out this prohibition.

References

- N4606 Working Draft, Standard for Programming Language C++

  http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4606.pdf

- llvm#29094 C++11 attributes not accepted in template explicit instantiation

  https://llvm.org/bugs/show_bug.cgi?id=29094